# Тема 9

• масиви от указатели към обекти

⊕
→ по-бързи swap-ове
→ не е нужен default-ен конструктор
→ nullptr позволява празни позиции
→ resize() не прави копия на обекти, а само мести пойнтери

⊖ → нема locality, понеже елементите на съседни позиции не са на съседни адреси в паметта

→ locality - при итериране през масив от обекти пази съседните в cache за по-бързо следващо достъпване

---

• типове стойности

→ lvalue - име на съществуваща променлива, ~~функция, стек данни~~, функция, връщаща референция (int& f() → f() = 7;)

→ rvalue = prvalue + xvalue

→ prvalue - литерали (6, nullptr, "ABC", true) и извиквания на функции, връщащи копие

→ xvalue (expiring value) - обект в края на живота си

```
A f() {...}
↓
f(), num++;
```

↳ xvalue, което съществува само временно, докато увеличи num

f(int) - приема lvalue и rvalue

f(int&) - приема lvalue

f(const int&) - приема lvalue и rvalue

f(int&&) - приема rvalue (std::move(lvalue) конвертира lvalue в xvalue)

↓

- rvalue reference – показва, че подадената променлива няма да се използва след функцията ⇒ можем да крадем нейните данни

- move семантики

→ не правим излишни копия, а крадем данните на другия обект, който и без това няма да бъде използван по-нататък

→ пестим време и памет

→ аналогично на copyFrom(const A&), изнасяме логиката по местенето на обекти във функция moveFrom(A&&)

→ move constructor и move op= в примера за MyString

noexcept

→ генерираният от компилатора т.с. вика т.с. на ~~композираните~~ вложените обекти, ако няма такъв, вика с.с., ако няма и такъв ⇒ ✗ (аналогично за move op=)

→ ако изтрием с.с. и не сме разписали наш т.с., компилаторът не генерира свой т.с.

```
...(A&& a) {
  // a се третира
  // като lvalue в f()
  // g(a); - ✗
  g(std::move(a)); // ✓
}
```

```
g(A&& a){}
```

---

```
f(string s){...}
```

↓

избираме с кой конструктор да създадем обекта

```
f(string("ABC"))   // 1x констр. 1x дестр.
f(str1)            // 1x к.к. 1x дестр.
f(std::move(str2)) // 1x м.с. 1x дестр.
```

---

• Пример за клас String с move семантика

```
#define st size_t
.h class String {
  public:
    String();
    String(const char*);
    String(const String&);
    String(String&&) noexcept;

    String& operator=(const String&);
    String& operator=(String&&) noexcept;

    ~String();

    ~~size_t~~ st getSize() const;
    const char* c_str() const;
    char& operator[](st index);
    const char& operator[](st index) const;
```